

The logo for Oracle Academy. The word "ORACLE" is in a bold, orange, sans-serif font. Below it, the word "Academy" is in a smaller, dark gray, sans-serif font. The entire logo is centered on a light gray background, which is framed by dark gray horizontal bars at the top and bottom.

ORACLE

Academy

Database Programming with PL/SQL

9-4

Managing Procedures and Functions

ORACLE
Academy



Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

Objectives

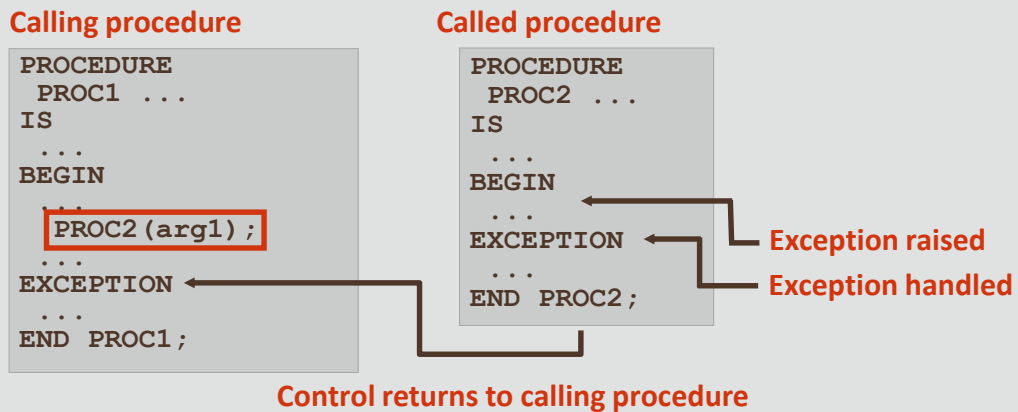
- This lesson covers the following objectives:
 - Describe how exceptions are propagated
 - Remove a function and a procedure
 - Use Data Dictionary views to identify and manage stored programs

Purpose

- In this lesson, you learn to manage procedures and functions
- To make your programs robust, you should always manage exception conditions by using the exception-handling features of PL/SQL

Handled Exceptions

- The following slides use procedures as examples, but the same rules apply to functions



When you develop subprograms that are called from other subprograms, you should be aware of the effects that handled and unhandled exceptions have on the transaction and the calling program.

When an exception is raised in a called procedure or function, the control immediately goes to the exception section of that block. An exception is considered handled if the exception section provides a handler for the exception raised.

Handled Exceptions

- When an exception occurs and is handled in a called procedure or function, the following code flow takes place:
 - 1. The exception is raised
 - 2. Control is transferred to the exception handler of the block that raised the exception
 - 3. The exception code is executed and the block is terminated
 - 4. Control returns to the calling program
 - 5. The calling program/block continues to execute

If a transaction was started (that is, if any data manipulation language [DML] statements executed before executing the called procedure in which the exception was raised), the transaction is unaffected (i.e., it is not automatically committed or rolled back).

However, if needed, you can explicitly end a transaction by executing a COMMIT or ROLLBACK operation in the exception section.

Handled Exceptions: Example

```
CREATE OR REPLACE PROCEDURE add_department(  
  p_name VARCHAR2, p_mgr NUMBER, p_loc NUMBER) IS  
BEGIN  
  INSERT INTO DEPARTMENTS (department_id,  
    department_name, manager_id, location_id)  
  VALUES (DEPARTMENTS_SEQ.NEXTVAL, p_name, p_mgr, p_loc);  
  DBMS_OUTPUT.PUT_LINE('Added Dept: ' || p_name);  
EXCEPTION  
  WHEN OTHERS THEN  
    DBMS_OUTPUT.PUT_LINE('Error adding dept: ' || p_name);  
END;
```

```
BEGIN  
  add_department('Media', 100, 1800);  
  add_department('Editing', 99, 1800);  
  add_department('Advertising', 101, 1800);  
END;
```



ORACLE
Academy

PLSQL 9-4
Managing Procedures and Functions

Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

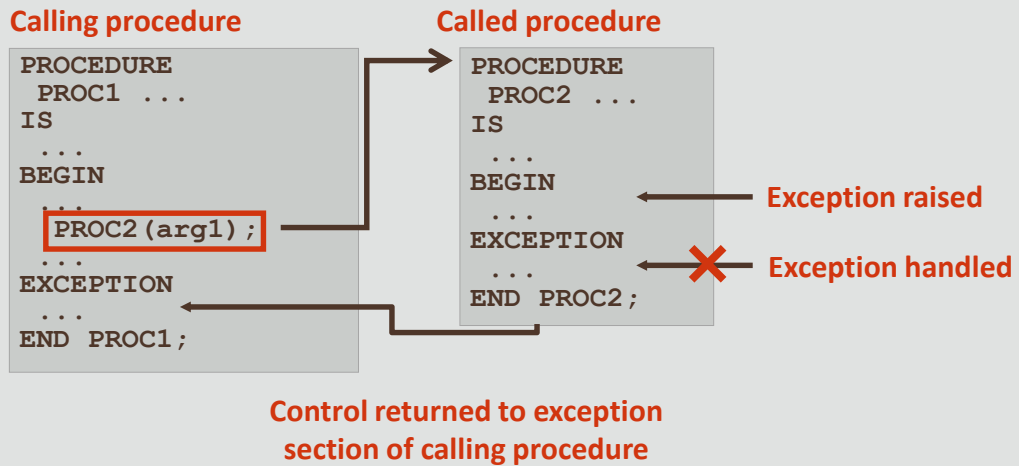
7

The ADD_DEPARTMENT procedure creates a new department record by allocating a new department number from an Oracle sequence, and sets the department_name, manager_id, and location_id column values using the arguments of the p_name, p_mgr, and p_loc parameters, respectively. The procedure catches all raised exceptions in its own handler.

The anonymous block makes three separate calls to the ADD_DEPARTMENT procedure.

The attempt to add an Editing department with manager_id of 99 is not successful because of a foreign key integrity constraint violation on the manager_id (manager_id must be a valid employee and there is no employee number 99 in the EMPLOYEES table). Because the exception was handled in the ADD_DEPARTMENT procedure, the anonymous block continues to execute. A query on the DEPARTMENTS table shows Media and Advertising were added, but not Editing.

Exceptions Not Handled



Exceptions Not Handled

- If the exception section does not provide a handler for the raised exception, then it is not handled
- The following code flow occurs:
 - 1. The exception is raised
 - 2. Control is transferred to the exception handler of the block that raised the exception
 - 3. Since no exception handler exists, the block terminates and any DML operations performed within the block that raised the exception are rolled back

Exceptions Not Handled

- If the exception section does not provide a handler for the raised exception, then it is not handled
- The following code flow occurs:
 - 4. Control returns to the calling program and the exception propagates to the exception section of the calling program
 - 5. If the exception is not handled by the calling program, the calling program terminates and any DML operations performed prior to the occurrence of the unhandled exception are rolled back

Exceptions Not Handled: Example

```
CREATE OR REPLACE PROCEDURE add_department_noex(  
    p_name VARCHAR2, p_mgr NUMBER, p_loc NUMBER) IS  
BEGIN  
    INSERT INTO DEPARTMENTS (department_id,  
        department_name, manager_id, location_id)  
    VALUES (DEPARTMENTS_SEQ.NEXTVAL, p_name, p_mgr, p_loc);  
    DBMS_OUTPUT.PUT_LINE('Added Dept: ' || p_name);  
END;
```

```
BEGIN  
    add_department_noex('Media', 100, 1800);  
    add_department_noex('Editing', 99, 1800);  
    add_department_noex('Advertising', 101, 1800);  
END;
```



• Result:

```
ORA-02291: integrity constraint (US_1217_S90_PLSQL.DEPT_MGR_FK) violated - parent  
key not found
```

ORACLE
Academy

PLSQL 9-4
Managing Procedures and Functions

Copyright © 2020, Oracle and/or its affiliates. All rights reserved.

11

The ADD_DEPARTMENT_NOEX procedure is identical to the ADD_DEPARTMENT procedure (see slide #5), except ADD_DEPARTMENT_NOEX does not have an exception section.

Running the anonymous block with calls to ADD_DEPARTMENT_NOEX results in the second call to ADD_DEPARTMENT_NOEX raising an exception that it does not handle. This causes the anonymous block to terminate without any departments being added to the DEPARTMENTS table.

When the anonymous block is executed:

1. The Media department is successfully inserted (but not committed).
2. The attempt to insert the Editing department raises an unhandled exception and returns control to the anonymous block, propagating the unhandled exception to the anonymous block.
3. The rest of the executable section of the anonymous block is skipped (so no attempt is made to insert the Advertising department) as control jumps to the exception section of the anonymous block.
4. Because the anonymous block does not handle the exception, the whole transaction is rolled back,

including the insert of the Media department.

Removing Procedures and Functions

- You can remove a procedure or function that is stored in the database
- Syntax:

```
DROP {PROCEDURE procedure_name | FUNCTION function_name}
```

- Examples:

```
DROP PROCEDURE my_procedure;
```

```
DROP FUNCTION my_function;
```

The subprogram must be in your own schema or you must have the DROP ANY PROCEDURE system privilege. If the subprogram you are dropping is called by any other stored subprogram, those subprograms will have their status changed to invalid and they will not recompile until the "error" is corrected (either the dropped subprogram is recreated or the call to it is removed).

Viewing Subprogram Names in the USER_OBJECTS Table

- This example lists the names of all the PL/SQL functions that you own:

```
SELECT object_name
FROM   USER_OBJECTS
WHERE  object_type = 'FUNCTION';  -- use 'PROCEDURE' to
                                   see procedures
```

OBJECT_NAME
TAX
DML_CALL_SQL



Viewing PL/SQL Source Code in the USER_SOURCE Table

- This example shows the source code of the TAX function, which you own
- Make sure you include ORDER BY line to see the lines of code in the correct sequence

```
SELECT text
FROM   USER_SOURCE
WHERE  name = 'TAX'
ORDER BY line;
```

TEXT
FUNCTION tax(value IN NUMBER)
RETURN NUMBER IS
BEGIN
RETURN (value*0.08);
END tax;

To see source code for subprograms owned by others (where you have EXECUTE privilege), use the following:

```
SELECT text
FROM   ALL_SOURCE
WHERE  name = 'TAX'
ORDER BY line;
```

Viewing Object Names and Source Code in Application Express

- You can easily view subprogram information in Application Express:
 - From SQL Workshop, click Object Browser, then Browse, and choose either Procedures or Functions as required
 - A list of subprograms appears
 - Click the required subprogram name
 - The source code of the subprogram appears
 - From here, you can edit and recompile it, or drop it if you want

Terminology

- Key terms used in this lesson included:
 - ALL_SOURCE
 - USER_OBJECTS
 - USER_SOURCE

- ALL_SOURCE – The table that contains source code for subprograms that are owned by others who have granted you the EXECUTE privilege.
- USER_OBJECTS – The table that contains the names and types of procedures and functions.
- USER_SOURCE – The table that contains source code for all of the subprograms that you own.

Summary

- In this lesson, you should have learned how to:
 - Describe how exceptions are propagated
 - Remove a function and a procedure
 - Use Data Dictionary views to identify and manage stored programs

The Oracle Academy logo is centered on a light gray background. It features the word "ORACLE" in a bold, orange, sans-serif font, with the letters "R" and "A" having a unique, slanted design. Below "ORACLE" is the word "Academy" in a dark gray, sans-serif font. The entire logo is framed by two dark gray horizontal bars, one at the top and one at the bottom.

ORACLE

Academy